

The curious case of an airport

Cláudia Faria (PG60240)
Patrícia Bastos (PG60287)

Universidade do Minho, Braga, Portugal
{PG60240, PG60287}@alunos.uminho.pt

April 30, 2026

1 Introduction

This project was done for the Cyber-Physical Programming class in the Formal Methods of Programming specialization of the Master's in Informatics Engineering at the University of Minho.

The goal is to implement a system in *Uppaal*, define liveness and safety properties that capture correct and fair behavior of that system and, finally, verify these properties.

2 Problem

A private airport is used by two planes, which can be either flying, parked, landing, or taking off. The airport is a resource shared by the two planes.

- **First stage:** Create an *Uppaal* model for the planes and the controller. The model should include 4 to 8 desired properties to be verified via *Uppaal*.
- **Second stage:** Extend the model to handle n planes at once and m runways at once.

3 Requirements and Properties

3.1 Initial requirements

The project statement introduced 8 requirements.

Req.	Description
1	Only one plane can use the runway at a time.
2	A controller component receives requests to <i>land</i> or to <i>take off</i> , and replies with a <i>wait</i> signal when the field is unavailable.
3	Each plane sends requests to the controller to <i>land</i> or to <i>take off</i> , and sends notifications when the field becomes <i>free</i> .
4	The controller has 5 time units to notify a plane to wait.
5	After 5 time units from requesting access to the field and with no wait signal, the planes take another 5 time units to reach the field.
6	Each plane takes non-deterministically between 1–3 time units to take off, and between 4–6 time units to land and park.
7	After taking off and after parking, the planes notify the controller with a <i>gone</i> signal.
8	If a plane is told to wait, it will take between 5–7 time units to request again.

Table 1: Initial Requirements

3.2 Desired properties

Assuming there are enough parking spaces for every plane, the **critical resource** in this problem is the runway of the airport. No two planes can be occupying the runway at once, as it presents the danger of a plane crash. Based on the given requirements and added safety constraints, the following desired properties were defined.

3.2.1 Reachability

Prop.	Description
1	Every plane can land.
2	Every plane can take off.

Table 2: Reachability properties

3.2.2 Safety

Prop.	Description
3	The system will never reach a deadlock.
4	There is never more than one plane in a runway.
5	If a plane is on the runway, then the controller must block access.

Table 3: Safety properties

3.2.3 Liveness

Prop.	Description
6	If a plane asks to take off, it will eventually be granted authorization to do so.
7	If a plane asks to land, it will eventually be granted authorization to do so.
8	If a plane starts takeoff, it eventually finishes.
9	If a plane starts landing, it eventually finishes.

Table 4: Liveness properties

4 Solution

In this solution, there are multiple states that a plane can be in:

- Parked
- Waiting to take off
- Field to take off
- Flying
- Waiting to land
- Field to land

Similarly, the controller can be in the following states:

- Free
- Busy

The first step of the model design was the interpretation of the requirements, and not all were directly adapted. An initial analysis of the requirements concluded that, for 2 planes, using both *gone* and *free* was redundant.

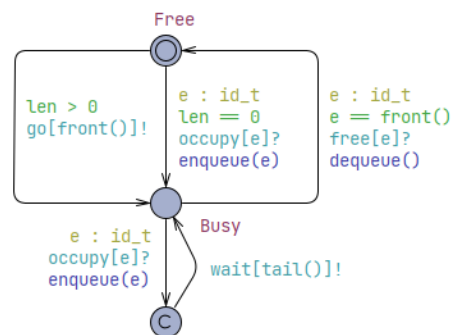
Additionally, during the construction of the model, the last requirement that requires planes to repeatedly request to land while being repeatedly told to wait was considered inefficient. To improve the system, this requirement was adapted into a **wait list** approach, which will be particularly helpful to later adapt to an arbitrary number of planes. The logic of the queue is *First In, First Out*. A plane is allowed to leave the wait list once it receives a *go* signal.

To simplify the communication, the *land* and *take off* requests were merged into *occupy*. The reasoning for this is that it does not matter to the controller why a plane is using the runway, only if it is available or not.

With the requirements interpreted, the second step was modelling the communication between the planes and the controller using *Calculus of Communicating Systems*. The following behaviour was defined.

- When the plane wants to access the runway, it sends an occupy signal to the controller. The plane will then have two separate possible paths: it will be ordered into the wait list until it received authorization to proceed, after which it will use the runway and inform the controller upon leaving it; or it will advance after not receiving any signal for 5 time units, informing the controller once the runway is available again.

- When the controller receives a request to occupy the runway, it will either: put the plane in a wait list, eventually grant it authorization to proceed and receive its confirmation that the runway is available again; or it will not order the plane to wait and eventually receive its confirmation that it has left the runway.



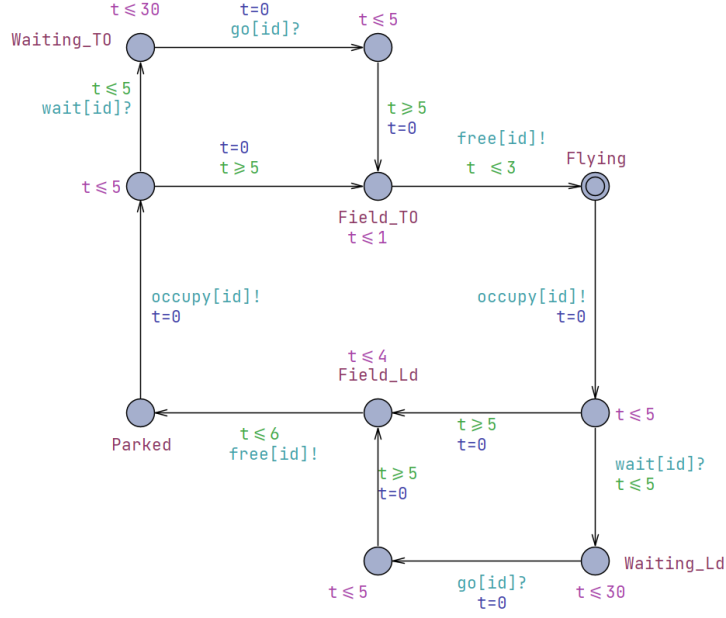


Figure 2: Plane Model

- **Parked/Flying to Waiting:** The plane sends a *occupy* signal to the controller, asking to occupy the runway. It then receives the instruction to *wait*, entering the wait list.
- **Waiting to Field:** The plane on the wait list receives authorization to use the runway, and proceeds to that state.
- **Parked/Flying to Field:** The plane sends a *occupy* signal to the controller, asking to occupy the runway. It does not receive an instruction to *wait* in under 5 time units, so it proceeds to the runway.
- **Field to Parked/Flying:** The plane sends a signal to the controller that the runway is available again, and continues on its journey.

Additional guards and invariants were added to fulfill the requirements that restrict the time it takes to perform take off as well as landing and parking. Specifically, the planes take non-deterministically between 1–3 time units to take off, and between 4–6 time units to land and park.

Some liveness properties that will be addressed in the next subsection were left unverified because of the possibility to remain in the Waiting state forever - essentially, a case of starvation. To mitigate this, a time limit of 30 time units was added to the Waiting state as an invariant. This means that a plane can only be made to wait, at most, 30 times units until it must allowed to enter the runway.

4.2 Verification of properties

Now the time has come to see if all of our desired properties can be verified. Using *Uppaal*'s Verifier, each property was checked to see if the system is functioning correctly.

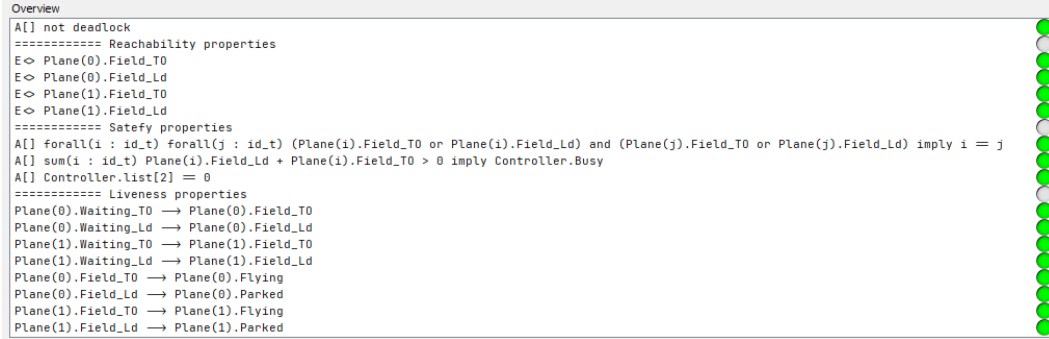


Figure 3: Testing Properties on the Verifier

Every property was satisfied. This means that our system is correct.

4.3 Extension to arbitrary case

To extend the system to an arbitrary number of planes and runways a Runway model was added and the Controller modified. The plane model remained the same as before.

The new controller can be in the following states:

- All runways are free
- Some runways are free
- All runways are busy

Similarly, the runway can be:

- Free
- Busy

In order to make the solution work for a number of N planes and M runways there was a need to define a waiting list in order to differentiate planes that are waiting for a *go* signal from the rest.

The following transitions were defined.

- **All Free to Some Free:** The controller receives a request to *occupy* and enqueues it, turning the state to Some Free.
- **Some Free to Some Free, occupying, partially filled queue and empty waiting list:** The controller receives another request to *occupy*, enqueues it.

- **Some Free to Some Free, leaving, queue with more than one plane:** A plane informs the controller that the runway is free, and is dequeued from the queue. Some runways are still occupied.
- **Some Free to All Free:** Only one plane is in the queue and the waiting list is empty. The plane informs the controller that the runway is free, and is dequeued from the queue. All runways are free.
- **Some Free to All Busy:** All runways are busy. The controller receives a request to *occupy*, enqueues it in the queue and in the waiting list, and tells it to wait.
- **All Busy to All Busy:** The controller receives a request to *occupy*, enqueues it in the queue and waiting list, and tells it to wait.
- **All Busy to Some Free:** A plane informs the controller that the runway is free, and is dequeued from the wait list. Some runways are still busy.

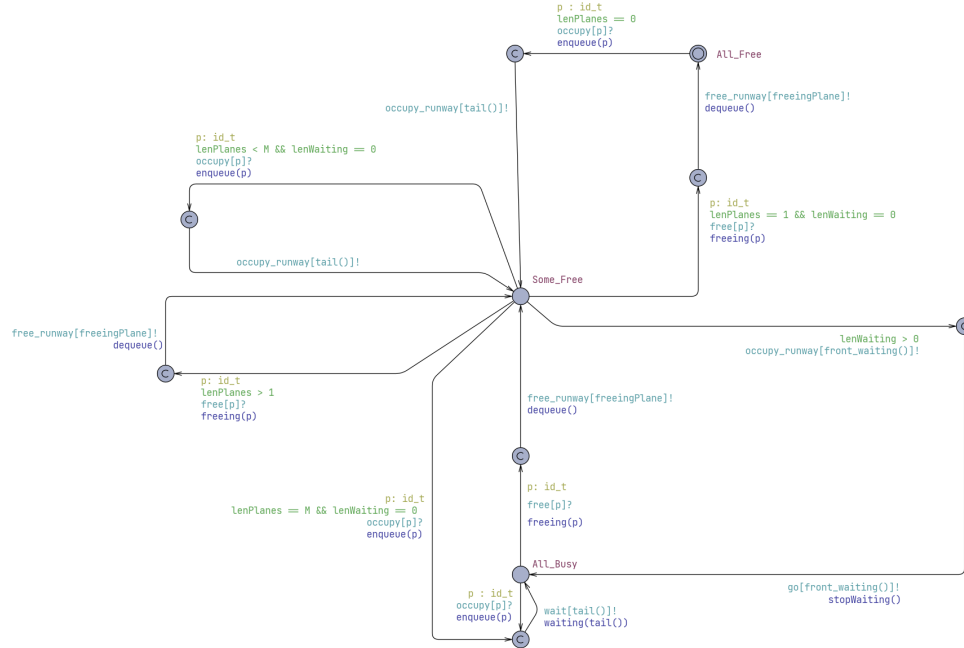


Figure 4: Controller Model for arbitrary runways and planes

A model was then produced for the runway using the defined states. The following transitions were defined.

- **Free to Busy:** The controller notifies the runway that it is gonna be occupied, it becomes busy and stores the plane occupying.
- **Busy to Free:** The controller notifies the runway it is no longer being used by said plane, it becomes free.



5 Conclusion

The first step to solve the problem of the Curious Case of an Airport was to derive requirements from the initial list. From those requirements, reachability, safety and liveness properties were defined to assure the correct functioning of our system. With the system's main components and logic defined, an *Uppaal* model was created with templates for the Controller and Plane. All properties were verified, proving the correctness of our system.

To extend the system to an arbitrary number of planes and runways, a template for the Runway was added and some states altered in the Controller to expand its complexity. Almost all properties were verified, with the exception of some liveness properties. This might imply that our system is suffering from rare cases of starvation, but the specific cause was not found.

This project allowed our group to learn more about system modelling and verification, and we consider our results to have been successful.